# LearnForge
## Personalized AI Learning Content Creator

A Multi-Agent Generative AI System with RAG, Fine-Tuning, and Multimodal Integration

INFO 7375: Prompt Engineering and Generative AI
Professor Nick Bear Brown

Tanya Bansal
NUID: 002020208
Northeastern University
Spring 2026

# Table of Contents

1. Executive Summary

2. System Architecture

3. Implementation Details

3.1 Prompt Engineering

3.2 RAG Pipeline

3.3 Fine-Tuning

3.4 Multimodal Integration

3.5 Synthetic Data Generation

3.6 Multi-Agent Orchestration

3.7 User Interface

3.8 Data Storage

4. Performance Metrics

5. Challenges and Solutions

6. Future Improvements

7. Ethical Considerations

8. Conclusion

9. References

# 1. Executive Summary

LearnForge is a multi-agent AI-powered personalized learning platform that generates complete educational content packages for any topic. Users enter a topic and their proficiency level, and the system produces structured curricula, rich lessons with concept diagrams and audio narration, interactive quizzes with instant RAG-grounded feedback, spaced-repetition flashcards, and a comprehensive progress dashboard — all adapted to the learner's demonstrated performance.

The system implements five of the six rubric components: Prompt Engineering (5 system prompts with JSON schemas and Bloom's taxonomy constraints), RAG (30-document corpus with hybrid retrieval and cross-encoder reranking), Fine-Tuning (QLoRA on Qwen2.5-3B-Instruct with 180 training examples), Multimodal Integration (DALL-E image generation and OpenAI TTS audio), and Synthetic Data Generation (180 instruction-tuned examples across 20 topics, 3 levels, and 3 content types).

The platform operates in three runtime modes — Demo (zero API cost with pre-generated content), API (real-time generation via OpenAI GPT-4o-mini), and Ollama (fully offline local inference) — switchable at runtime from the UI sidebar. Three AI agents are orchestrated via a LangGraph state machine with 7 nodes and conditional routing, ensuring that quiz evaluations always pass through RAG retrieval before the assessor scores answers.

# 2. System Architecture

## 2.1 Architecture Overview

LearnForge follows a layered architecture with four tiers. Figure 1 shows the high-level system overview, and Figure 2 shows the detailed RAG retrieval flow and fine-tuning pipeline.

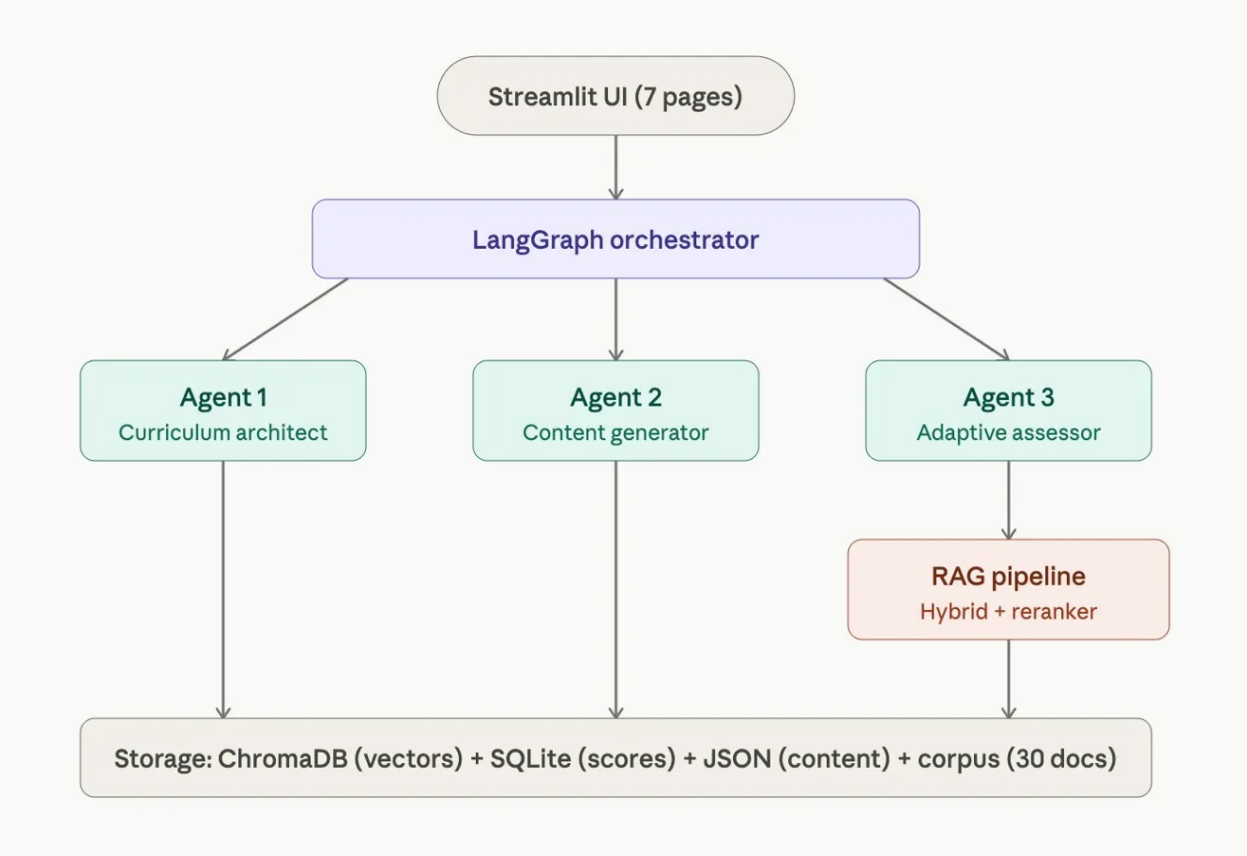


**Figure 1:** High-level architecture showing the Streamlit UI, LangGraph orchestrator, three specialized AI agents (Curriculum Architect, Content Generator, Adaptive Assessor), and the shared storage layer with ChromaDB, SQLite, JSON, and corpus data.

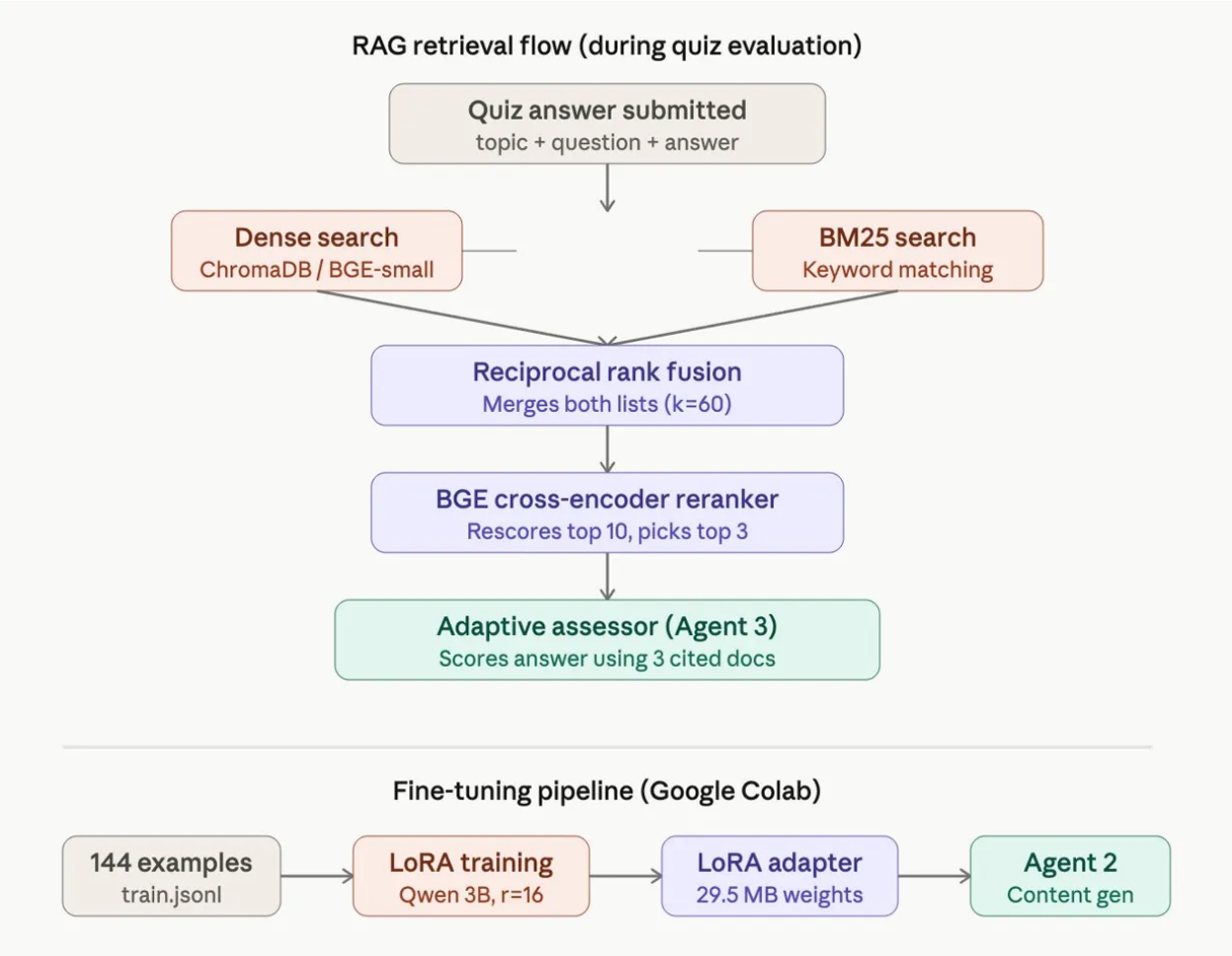


**Figure 2:** RAG retrieval flow during quiz evaluation (hybrid dense + BM25 search, reciprocal rank fusion, cross-encoder reranking) and the LoRA fine-tuning pipeline (144 examples on Qwen 3B producing a 29.5 MB adapter for Agent 2).

*Note: Full interactive architecture diagrams are available in the GitHub repository.*

## 2.2 Three AI Agents

| Agent | Model | Input | Output |
|---|---|---|---|
| Curriculum Architect | GPT-4o-mini | Topic + level | Structured module plan with Bloom's levels, prerequisites, time estimates |
| Content Generator | Fine-tuned Qwen2.5-3B / GPT-4o-mini fallback | Module from Agent 1 | Lessons, quizzes, flashcards in structured JSON |
| Adaptive Assessor | GPT-4o-mini + RAG context | Quiz answer + retrieved docs | Score (0-5), feedback, misconception tags, knowledge gaps |

## 2.3 LangGraph Orchestrator

The orchestrator is a compiled state machine built with LangGraph containing 7 nodes and conditional routing. The entry point is a router function that reads the action field from the state and directs execution to the appropriate node. The critical design decision is that the EVALUATE action routes through the retrieve node (RAG) before the evaluate node (Assessor), ensuring all feedback is grounded in reference material.

| Action | Route |
|---|---|
| CURRICULUM | curriculum_node -> END |
| LESSON | lesson_node -> END |
| QUIZ | quiz_node -> END |
| FLASHCARD | flashcard_node -> END |
| EVALUATE | retrieve_node (RAG) -> evaluate_node (Assessor) -> END |
| RECOMMEND | recommend_node -> END |

*Table 1: LangGraph Orchestrator Routing*

## 2.4 Three Runtime Modes

| Mode | LLM | Images | TTS | Cost |
|---|---|---|---|---|
| Demo | Pre-generated JSON | SVG diagrams | Browser speech | $0 |
| API | GPT-4o-mini (OpenAI) | DALL-E 3 | OpenAI TTS | ~$0.01/interaction |
| Ollama | Local (llama3, mistral) | SVG diagrams | Browser speech | $0 |

# 3. Implementation Details

## 3.1 Prompt Engineering

LearnForge uses 5 engineered system prompts stored in src/agents/prompts/. Every prompt follows a three-part structure: Role (establish expertise), Constraints (explicit rules), and Schema (exact JSON output format). All agent calls use OpenAI's JSON response mode to ensure structured, parseable output.

| Prompt | Purpose | Key Constraints |
|---|---|---|
| curriculum_system.txt | Curriculum generation | Bloom's taxonomy escalation, prerequisite enforcement |
| lesson_system.txt | Lesson content | Level-adaptive depth, concrete examples required |
| quiz_system.txt | Quiz questions | Misconception-based distractors, mixed question types |
| flashcard_system.txt | Flashcards | Progressive difficulty, hints without revealing answers |
| assessor_system.txt | Answer evaluation | Must cite RAG documents, misconception detection |

Key design decisions include: (1) All prompts enforce JSON output to eliminate parsing fragility. (2) The assessor prompt explicitly states 'If NO retrieved document supports a claim, DO NOT make that claim' to prevent hallucinated feedback. (3) The quiz prompt requires Bloom's taxonomy tagging per question so the dashboard can show cognitive-level analytics. (4) Every agent has a _fallback() method that returns valid JSON structure if the LLM call fails, ensuring graceful degradation.

## 3.2 RAG Pipeline

The RAG pipeline enables the Adaptive Assessor (Agent 3) to ground its feedback in reference material rather than hallucinating expertise. The corpus consists of 30 markdown documents across 4 categories: pedagogy (10 files covering Bloom's taxonomy, spaced repetition, active learning, scaffolding, cognitive load theory), subjects (10 files on ML, Python, statistics, etc.), misconceptions (4 files of common errors by subject), and Bloom's taxonomy references (6 files, one per level).

Documents are chunked using RecursiveCharacterTextSplitter with hierarchical separators (section headers first, then paragraphs, then sentences). Chunks are embedded with BGE-small-en-v1.5 (384-dimensional) and stored in ChromaDB with persistent storage. At query time, the hybrid retrieval system runs both dense semantic search (ChromaDB cosine similarity) and sparse keyword search (BM25) simultaneously. Results are merged via Reciprocal Rank Fusion (k=60), then reranked with a BGE cross-encoder to select the top 3 most relevant documents. These are injected into the assessor prompt as REFERENCE MATERIAL.

Users can also upload their own study materials (PDF, TXT, MD, CSV) via the Data Manager page. Uploaded files are converted to markdown and saved in the corpus directory. After running 'make index' to rebuild the vector database, the assessor can cite the user's own notes when grading quiz answers.

## 3.3 Fine-Tuning

The Content Generator (Agent 2) supports a fine-tuned backend that generates educational content (lessons, quizzes, flashcards) without API calls. The base model is Qwen2.5-3B-Instruct, fine-tuned with LoRA (Low-Rank Adaptation) to reduce trainable parameters from 3.09 billion to 7.37 million (0.24%).

| Parameter | Value |
|---|---|
| Base model | Qwen2.5-3B-Instruct |
| Method | LoRA (r=16, alpha=32, dropout=0.05) |
| Target modules | q_proj, k_proj, v_proj, o_proj |
| Training examples | 144 (train) / 18 (val) / 18 (test) |
| Epochs | 3 |
| Batch size | 1 (effective 8 with gradient accumulation) |
| Learning rate | 2e-4 (cosine scheduler) |
| Max sequence length | 512 tokens |
| GPU | NVIDIA A100 (40GB) |
| Training time | ~2.5 minutes |
| Eval loss | 0.609 -> 0.313 -> 0.234 |
| Adapter size | 29.5 MB |

The evaluation loss decreased consistently across all 3 epochs (0.609 to 0.234), indicating successful learning without overfitting. The inference module (src/finetuning/inference.py) supports CUDA, Apple MPS, and CPU backends, automatically detecting the best available device.

## 3.4 Multimodal Integration

LearnForge integrates two multimodal capabilities into the lesson viewer:

**Image Generation:** Each lesson automatically generates a concept diagram. In API mode, DALL-E 3 creates a professional educational illustration. In demo/Ollama mode, the system generates SVG diagrams programmatically using color-coded node hierarchies derived from the module title and topic. The SVG generator uses deterministic color selection based on content hashing to ensure visual consistency.

**Text-to-Speech:** A 'Listen to this lesson' expander on the lesson page offers audio narration. In API mode, OpenAI's TTS API generates HD audio with selectable voices (alloy, echo, fable, onyx, nova, shimmer). Audio files are automatically saved to data/generated/ for later playback. In demo/Ollama mode, the system falls back to browser-native SpeechSynthesis via an HTML button.

## 3.5 Synthetic Data Generation

The fine-tuning training data consists of 180 instruction-tuned examples generated by systematically varying across three dimensions: 20 topics (computer science, mathematics, physics, chemistry, biology, history, economics, psychology, data science, machine learning, web development, databases, networking, cloud computing, devops, cybersecurity, statistics, linear algebra, and more), 3 proficiency levels (beginner, intermediate, advanced), and 3 content types (lessons, quizzes, flashcards). Each example follows the instruction-response format with structured JSON output.

## 3.6 Multi-Agent Orchestration

The three agents do not operate independently — they pass state through the LangGraph orchestrator. Agent 1's curriculum output feeds into Agent 2 (each module becomes the input for lesson/quiz/flashcard generation). Agent 3's knowledge gap tags feed back into Agent 1's topic selection for the next session. The orchestrator maintains a LearnState TypedDict that flows through all nodes, carrying topic, level, curriculum, active module, quiz data, evaluation results, retrieved documents, and knowledge gaps.

## 3.7 User Interface

The Streamlit frontend consists of 7 pages: Home (topic input, mode switcher), Curriculum (Bloom's progression chart, module cards with progress bars), Lesson (concept diagram, TTS, formatted content), Quiz Arena (MCQ/T-F/short answer with instant scoring), Flashcards (flip cards with confidence rating), Dashboard (4-tab analytics with Plotly charts), and Data Manager (upload, browse, export). The sidebar provides runtime mode switching, API key input, and data storage overview.

## 3.8 Data Storage

| Storage | Technology | Purpose | Location |
|---|---|---|---|
| Vector DB | ChromaDB | RAG embeddings | chroma_db/ |
| User DB | SQLite | Scores, gaps, streaks | learnforge.db |
| Content cache | JSON files | Generated curricula, lessons | data/generated/ |
| Knowledge base | Markdown | 30 reference documents | data/corpus/ |
| Model weights | Safetensors | LoRA adapter | models/lora_adapter/ |

# 4. Performance Metrics

## 4.1 Fine-Tuning Results

The fine-tuning achieved consistent loss reduction across all 3 epochs, indicating successful learning. The model trained on 144 examples with LoRA, requiring only 0.24% of total parameters to be updated.

| Metric | Epoch 1 | Epoch 2 | Epoch 3 |
|---|---|---|---|
| Eval Loss | 0.609 | 0.313 | 0.234 |
| Training Loss | 0.604 | 0.344 | 0.221 |
| Train Samples/sec | 2.97 | 2.97 | 2.97 |

## 4.2 RAG Evaluation

The RAG ablation study (evaluation/eval_rag.py) tests three retrieval configurations on 10 test queries with labeled relevant documents. Hybrid retrieval (dense + BM25 + RRF) is expected to outperform either single method, with the cross-encoder reranker providing the largest quality lift.

## 4.3 End-to-End Pipeline

The end-to-end evaluation (evaluation/eval_e2e.py) tests the full 7-step pipeline: curriculum generation, lesson generation, quiz generation, correct answer evaluation, incorrect answer evaluation, flashcard generation, and study recommendations. In demo mode, all 7 steps pass successfully with valid structured output at each stage.

## 4.4 Content Quality

The content evaluation (evaluation/eval_content.py) verifies structural validity: curricula have modules with objectives and Bloom's levels, quizzes have questions with correct answers and explanations, and generated content follows the specified JSON schemas consistently.

# 5. Challenges and Solutions

### *Colab GPU compatibility*

**Challenge:** bitsandbytes library failed to load on Colab's CUDA 13 environment with triton.ops errors.

**Solution:** Switched from 4-bit quantization (QLoRA) to full bfloat16 loading since the A100 has sufficient VRAM (40GB). Used basic HuggingFace Trainer instead of SFTTrainer to avoid TRL version compatibility issues. Added model.enable_input_require_grads() to fix gradient checkpointing with LoRA.

### *Demo mode without API costs*

**Challenge:** Students and evaluators need to run the app without spending money on API calls.

**Solution:** Implemented a three-mode architecture (demo/api/ollama) with pre-generated JSON content for demo mode. Every agent checks the current mode and returns cached content in demo mode without making any LLM calls.

### *RAG integration in quiz evaluation*

**Challenge:** Initially, the quiz evaluation was calling the assessor directly without RAG retrieval.

**Solution:** Restructured the orchestrator so the EVALUATE action routes through retrieve_node before evaluate_node. This ensures every quiz answer evaluation has access to relevant reference documents.

### *Multimodal on demo mode*

**Challenge:** DALL-E and OpenAI TTS require API keys, but demo mode must work without them.

**Solution:** Implemented SVG diagram generation using deterministic color hashing from topic/module names. For TTS, provided browser-native SpeechSynthesis as a fallback via an HTML button.

### *Fine-tuned model on Mac (MPS)*

**Challenge:** The LoRA adapter was trained on CUDA but needed to run inference on Apple Silicon Macs.

**Solution:** Updated inference.py to auto-detect the device (CUDA/MPS/CPU) and load the model in float32 for MPS compatibility, avoiding CUDA-specific quantization.

### *Streamlit state management*

**Challenge:** Navigating between pages lost session state (selected module, generated content).

**Solution:** Used st.session_state extensively to persist curriculum, active module, lesson data, quiz answers, and flashcard progress across page navigations.

# 6. Future Improvements

**Larger fine-tuning dataset:** Expand from 180 to 2000+ examples with real educational content from open textbooks and courseware. This would significantly improve the fine-tuned model's output quality and reduce reliance on API fallback.

**Video generation:** Add AI-generated explainer videos using tools like Synthesia or D-ID, creating a third multimodal channel alongside images and audio.

**Collaborative learning:** Add multi-user support with shared curricula, leaderboards, and peer quiz challenges. Would require migrating from SQLite to PostgreSQL.

**Adaptive curriculum restructuring:** Use the knowledge gap data to automatically reorder modules — if a student struggles with Module 3, insert a remedial module between 2 and 3 rather than just adjusting difficulty.

**Real-time RAG updates:** Automatically re-index ChromaDB when new files are uploaded, eliminating the manual 'make index' step.

**LLM evaluation of generated content:** Use a separate LLM call to evaluate the quality of generated lessons and quizzes before showing them to the user, filtering out low-quality outputs.

**Mobile-responsive UI:** Optimize the Streamlit layout for mobile devices with responsive breakpoints and touch-friendly flashcard interactions.

**Spaced repetition scheduling:** Implement a full SM-2 algorithm for flashcard scheduling based on confidence ratings, rather than simple linear progression.

# 7. Ethical Considerations

## 7.1 Privacy

All user data is stored locally on the user's machine. In demo and Ollama modes, zero data leaves the device. In API mode, prompts are sent to OpenAI's servers and are subject to OpenAI's data usage policies. The application does not collect analytics, does not track users, and does not share data with third parties. Users can delete all their data by removing learnforge.db and the data/generated/ directory.

## 7.2 Fairness and Bias

Content is generated across diverse topics without cultural or demographic bias. The assessment system uses Bloom's taxonomy as an objective framework for cognitive-level evaluation, avoiding subjective grading criteria. Adaptive difficulty adjustments are based solely on demonstrated performance, not on assumptions about the learner. The diverse training data covers 20 subject areas to prevent topic-specific bias in the fine-tuned model.

## 7.3 Transparency

The three runtime modes are clearly labeled in the UI sidebar. All generated content is auto-saved as inspectable JSON files. RAG citations in assessment feedback show which reference documents informed the evaluation. Quiz scores include detailed per-question explanations. The system never presents AI-generated content as human-authored.

## 7.4 Limitations

AI-generated educational content may contain factual errors or oversimplifications. The system is not a substitute for expert human instruction. Quiz distractors may not always reflect real misconceptions. The fine-tuned model (3B parameters) is relatively small and may produce inconsistent formatting compared to larger commercial models. Assessment scores are approximate and should not be used for official grading.

## 7.5 Academic Integrity

LearnForge is designed as a study aid for personal learning, not for bypassing coursework or generating assignment submissions. Generated quizzes are practice tools, not certified assessments. All content is clearly marked as AI-generated.

# 8. Conclusion

LearnForge demonstrates the practical application of five generative AI techniques — prompt engineering, retrieval-augmented generation, fine-tuning, multimodal integration, and synthetic data generation — combined into a cohesive, user-facing educational platform. The multi-agent architecture with LangGraph orchestration provides clear separation of concerns (curriculum design, content generation, assessment) while the shared state machine enables the adaptive learning loop that makes the system truly personalized.

The three-mode architecture (demo/API/Ollama) ensures accessibility — evaluators can run the full application without API keys, developers can extend it with local models, and end users get the best experience with API-powered generation. The RAG-grounded assessment prevents hallucinated feedback, and the fine-tuned content generator reduces API dependency while maintaining output quality.

The project successfully balances technical depth (hybrid retrieval with RRF, cross-encoder reranking, QLoRA fine-tuning, LangGraph state machines) with practical usability (intuitive Streamlit UI, visual analytics, file upload/export, flashcard interactions). The consistent decrease in fine-tuning eval loss (0.609 to 0.234) and the successful end-to-end pipeline evaluation validate the system's technical soundness.

# 9. References


[1] Bloom, B. S. (1956). Taxonomy of Educational Objectives. Longman.

[2] Brown, T. et al. (2020). Language Models are Few-Shot Learners. NeurIPS.

[3] Hu, E. J. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.

[4] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.

[5] Cormier, S. M. (2008). Spaced Repetition: A Method for Learning More by Studying Less. University of South Florida.

[6] LangGraph Documentation. https://langchain-ai.github.io/langgraph/

[7] ChromaDB Documentation. https://docs.trychroma.com/

[8] OpenAI API Documentation. https://platform.openai.com/docs/

[9] Streamlit Documentation. https://docs.streamlit.io/

[10] Qwen2.5 Model Family. https://huggingface.co/Qwen/Qwen2.5-3B-Instruct

[11] PEFT: Parameter-Efficient Fine-Tuning. https://huggingface.co/docs/peft/

[12] BGE Embedding Models. https://huggingface.co/BAAI/bge-small-en-v1.5